

## Estruturas de Dados

### Trabalho 02 - *Processamento de Expressões Matemáticas*

Prof. Raul de Araújo Lima

**Disciplina:** Estruturas de Dados — Sistemas de Informação

**Linguagem:** Python 3 (apenas biblioteca padrão) · **Organização:** em duplas

**Entrega:** 02/10/2026 · **Apresentação:** 05/10/2026

## 1. O problema

Quando você digita  $3 + 4 * 2$  em uma calculadora, em uma planilha ou em um programa, alguma coisa precisa transformar essa sequência de caracteres em um número. Compiladores, interpretadores e bancos de dados fazem isso o tempo todo.

Parece simples, mas não é. Leia a expressão da esquerda para a direita, um símbolo por vez, e tente calcular assim que puder:

```
3      guardo o 3
+      ...e agora? não sei com o que somar
4      seria 4? ou vem algo depois?
*      de fato vinha: o + não é 3 + 4, mas sim 3 + 8...
2
```

O problema é que, na notação que usamos, **ler um operador não significa poder aplicá-lo**. O que vem depois pode mudar a que ele se aplica. A precedência e os parênteses só fazem sentido quando a expressão inteira já foi lida.

### 1.1. Por que uma pilha resolve isso

Se você não pode aplicar o operador agora, precisa **guardá-lo e retomá-lo depois**. A pergunta seguinte é: retomar em que ordem?

Em  $3 + 4 * 2$ , o  $*$  foi escrito **depois** do  $+$ , mas precisa ser resolvido **antes** dele. Em  $(3 + 4) * 2$ , o parêntese inverte essa ordem. O que se repete nos dois casos é que a decisão sempre envolve o item **mais recente** que ficou pendente.

Guardar itens e retomar o mais recente primeiro é exatamente o contrato da pilha. É por isso que ela aparece aqui — não por conveniência de implementação, mas porque a estrutura do problema é essa.

### 1.2. Os dois desafios

Uma forma de atacar o problema é dividi-lo em duas etapas independentes, cada uma com sua própria pilha:

|   | Desafio                                  | Entrada   | Saída     |
|---|------------------------------------------|-----------|-----------|
| 1 | Avaliar expressão em notação pós-fixa    | 3 4 2 * + | 11        |
| 2 | Converter da notação usual para pós-fixa | 3 + 4 * 2 | 3 4 2 * + |

**Resolvam nessa ordem. O Desafio 2 produz exatamente a entrada que o Desafio 1 consome, e o programa final encadeia os dois.**

Em cada desafio, a pilha guarda uma coisa diferente. Descobrir **o quê** faz parte do trabalho.

## 2. Desafio 1 — Avaliar expressão pós-fixa

Na notação pós-fixa (também chamada polonesa reversa), o operador vem **depois** dos seus operandos:

| Notação usual | Pós-fixa  |
|---------------|-----------|
| 3 + 4         | 3 4 +     |
| 3 + 4 * 2     | 3 4 2 * + |
| (3 + 4) * 2   | 3 4 + 2 * |

Repare no que desapareceu: **não há parênteses e não há regra de precedência**. A ordem dos símbolos já resolveu toda a ambiguidade. É por isso que essa notação é usada internamente por calculadoras e máquinas virtuais — avaliá-la é direto.

Escrevam um programa que leia expressões pós-fixas, com os tokens separados por espaço, e calcule o resultado. Operadores aceitos: + - \* / ^.

Testem com - e / desde cedo. Expressões só com + e \* escondem um erro muito comum de ordem de operandos.

### 2.1. Saída exigida

Não basta imprimir o resultado. Para cada expressão, imprimam o conteúdo da pilha **após** processar cada token:

```
token | pilha
3      | 3
4      | 3 4
+      | 7
resultado: 7
```

O token ocupa 5 caracteres alinhados à esquerda, seguido de | e da pilha listada da **base para o topo**, separada por espaços.

Para que não haja divergência na impressão de números, usem esta função em todos os valores:

```
def fmt(v):
    """Imprime 8 em vez de 8.0, mas mantém 2.5 como 2.5."""
    return f"{v:g}"
```

A formatação :g é a "geral" do Python: remove zeros desnecessários à direita. `fmt(8.0)` devolve '8'; `fmt(2.5)` devolve '2.5'.

### 2.2. Expressões inválidas

O programa não pode encerrar com exceção não tratada. Ao detectar um problema, imprima a mensagem no lugar da linha resultado: e siga para a próxima expressão.

| Situação                | Exemplo | Mensagem                      |
|-------------------------|---------|-------------------------------|
| Operandos insuficientes | 3 + * 4 | erro: operandos insuficientes |
| Operandos sobrando      | 3 4 5 + | erro: expressao malformada    |
| Divisão por zero        | 8 0 /   | erro: divisao por zero        |

## 2.3. Casos para conferir

| Entrada           | Resultado |
|-------------------|-----------|
| 3 4 +             | 7         |
| 3 4 2 * +         | 11        |
| 3 4 + 2 *         | 14        |
| 10 3 -            | 7         |
| 20 4 /            | 5         |
| 5 1 2 + 4 * + 3 - | 14        |

## 3. Desafio 2 — Converter para pós-fixa

Agora a entrada é a notação usual, com parênteses e precedência:  $3 + 4 * (2 - 1)$ . Produzam a pós-fixa equivalente e, em seguida, calculem-na reaproveitando o Desafio 1.

Esta é a etapa em que a ambiguidade da notação precisa ser resolvida, e é aqui que a pilha faz o trabalho pesado.

### 3.1. Regras da notação (especificação)

| Operador | Precedência    | Associatividade |
|----------|----------------|-----------------|
| + -      | 1 (mais fraca) | esquerda        |
| * /      | 2              | esquerda        |
| ^        | 3 (mais forte) | direita         |

Associar à esquerda significa que  $10 - 3 - 2$  equivale a  $(10 - 3) - 2$ , resultando em 5. Associar à direita significa que  $2 ^ 3 ^ 2$  equivale a  $2 ^ (3 ^ 2)$ , resultando em **512** — e não 64. Os parênteses continuam valendo e têm prioridade sobre tudo.

### 3.2. Tokenização

A entrada pode vir sem espaços entre os símbolos:

```
3+42*(7-1)
```

Antes de converter, é preciso separar a expressão em tokens, reconhecendo números de **mais de um dígito** e decimais. O resultado esperado para a linha acima é:

```
['3', '+', '42', '*', '(', '7', '-', '1', ')']
```

Este sub-problema **não** se resolve com pilha, e também não se resolve com `split()`. Pensem no que distingue um caractere que continua o número anterior de um que inicia um token novo.

### 3.3. Saída exigida

Para cada expressão, imprimam o estado da pilha de operadores e da saída **após** cada token, encerrando com a pós-fixa obtida:

|       |       |       |
|-------|-------|-------|
| token | pilha | saída |
| 3     | -     | 3     |
| +     | +     | 3     |
| 4     | +     | 3 4   |
| (fim) | -     | 3 4 + |

posfixa: 3 4 +

Token em 5 caracteres, pilha em 11, ambos alinhados à esquerda, separados por `|`. Usem `-` quando a pilha ou a saída estiverem vazias. A última linha do rastreamento usa o token literal `(fim)`, representando o esvaziamento final.

Em seguida, imprimam o rastreamento do Desafio 1 sobre a pós-fixa obtida, no formato já definido.

### 3.4. Expressões inválidas

| Situação                  | Exemplo | Mensagem                              |
|---------------------------|---------|---------------------------------------|
| Fechamento sem abertura   | 3 + 4)  | erro: parentese fechado sem abertura  |
| Abertura sem fechamento   | (3 + 4  | erro: parentese aberto sem fechamento |
| Caractere não reconhecido | 3 \$ 4  | erro: caractere invalido              |

### 3.5. Casos para conferir

| Entrada           | Pós-fixa          | Resultado |
|-------------------|-------------------|-----------|
| 3 + 4 * 2         | 3 4 2 * +         | 11        |
| (3 + 4) * 2       | 3 4 + 2 *         | 14        |
| 1 + 2 * 3 - 4 / 2 | 1 2 3 * + 4 2 / - | 5         |
| 3 + 4 * (2 - 1)   | 3 4 2 1 - * +     | 7         |
| 10 - 3 - 2        | 10 3 - 2 -        | 5         |
| 2 ^ 3 ^ 2         | 2 3 2 ^ ^         | 512       |
| (2 ^ 3) ^ 2       | 2 3 ^ 2 ^         | 64        |

As três últimas linhas verificam a associatividade, e falham de formas diferentes: quem tratar **todos** os operadores como associativos à esquerda obtém 64 em  $2 \wedge 3 \wedge 2$ ; quem tratar **todos** como associativos à direita obtém 9 em  $10 - 3 - 2$ . Só a combinação correta acerta as três.

## 4. Exigências de implementação

1. Usem a classe Pilha (encadeada) construída em aula, respeitando seu contrato: empilhar, desempilhar, topo, vazia, tamanho.
2. Não acessem a lista interna da pilha, não usem `lista.pop(0)` e não substituam a pilha por fatiamento de lista.
3. O rastreamento precisa ler o conteúdo da pilha **sem destruí-lo**. Se for necessário percorrê-la, acrescentem um método para isso na própria classe.
4. Separem a lógica em funções distintas — tokenizar, converter e avaliar. Nenhuma delas deve imprimir nada; a impressão fica por conta do programa principal.

## 5. Entrega

Entreguem um arquivo compactado contendo:

| Arquivo     | Conteúdo                                                                           |
|-------------|------------------------------------------------------------------------------------|
| desafio1.py | Avaliação de expressões pós-fixas                                                  |
| desafio2.py | Conversão de expressões convencionais em pós-fixas (recebe string, retorna string) |
| main.py     | Utiliza o processo dos dois desafios pra resolver expressões convencionais         |
| registro.md | Um readme do projeto apresentando a solução desenvolvida                           |

## 6. Duplas ou trios e apresentação

O trabalho é feito em **duplas ou trios**. Apenas um dos dois envia o arquivo (.zip) pelo AVA, com **nome e matrícula dos integrantes** identificados no envio e no início de cada arquivo Python e no registro.md.

Haverá uma **apresentação presencial por equipe**, em sala, na data indicada no cabeçalho. Os integrantes entram juntos e apresentam o que desenvolveram. Serão feitas perguntas sobre as decisões de implementação, **dirigidas individualmente**, e poderá ser solicitada uma pequena modificação no código durante a apresentação.

## 7. Avaliação

| Componente                                           | Peso |
|------------------------------------------------------|------|
| Rastreamentos e resultados corretos (Desafios 1 e 2) | 25%  |
| Apresentação e modificação em sala                   | 50%  |
| Tratamento das expressões inválidas                  | 15%  |
| Registro do trabalho (registro.md)                   | 10%  |

O código funcionar é pré-requisito, não a maior parte da nota. O que pesa é a capacidade de **explicar e modificar** o que foi entregue.

**Registro do processo.** No arquivo `registro.md`, incluam um “readme” em Markdown do projeto, explicando o código, as funções e estruturas utilizadas. No início desse arquivo deve aparecer o nome e matrícula dos integrantes da equipe.

**Uso de ferramentas de IA.** Permitido como ferramenta de estudo, desde que declarado ao final do `registro.md`, informando quais ferramentas foram usadas e para quê. Declarar não reduz a nota; omitir configura desonestidade acadêmica. A nota vem da apresentação, não do código entregue.